

# RAG (Retrieval-Augmented Generation) & Advanced RAG

## LLM의 Limitation

- Inference
  1. Domain-specific 또는 knowledge-intensive tasks에 약함.
    - ※ Knowledge-intensive: 외부 정보를 사용하는 NLP task
      - Open-domain QA
      - Fact verification
  2. 학습 종료 이후 시점의 지식이 없음. (Training cutoff)
  3. Real-time 정보 처리 불가능.
  4. 답변의 출처/근거를 알 수 없음.
- ⚠ 환각 (Hallucination)
- Fine-tuning with New Dataset
  1. Training cost
  2. LLM이 long-tail 정보 전체를 parameters에 완벽히 기억하도록 하는 것은 불가능.
  3. 학습한 최신 정보가 금방 outdated됨.
  4. 보안 문제 (e.g. LLM이 기업/기관의 내부 정보를 기억하는 것을 원치 않을 때)
  5. 답변의 출처/근거를 알 수 없음.

✅ LLM + Retrieval: RAG

## RAG 개념의 등장

Title: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Venue: NeurIPS 2020

Link:

<https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>

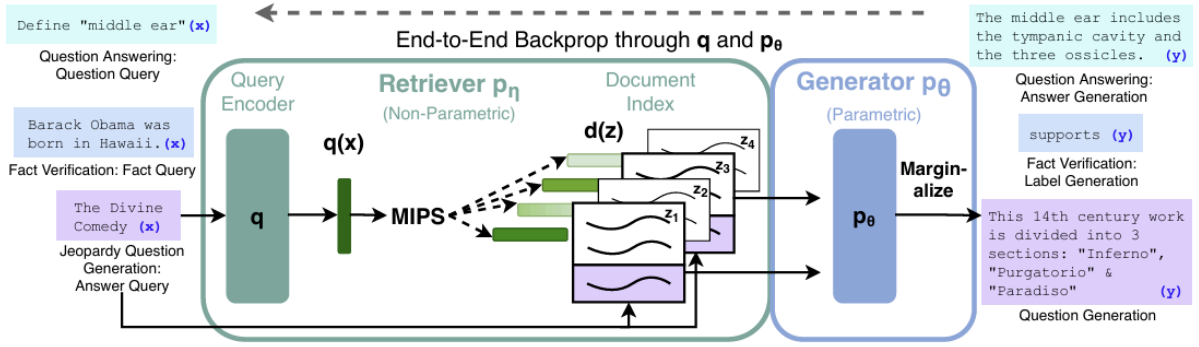


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder + Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query  $x$ , we use Maximum Inner Product Search (MIPS) to find the top-K documents  $z_i$ . For final prediction  $y$ , we treat  $z$  as a latent variable and marginalize over seq2seq predictions given different documents.

## 1. Methods

$$p_{\eta}(z|x) \propto \exp(\mathbf{d}(z)^{\top} \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{BERT}_d(z), \quad \mathbf{q}(x) = \text{BERT}_q(x)$$

- Retriever: DPR
  - 모든 외부 document  $z \in \{z_1, z_2, \dots\}$ 를 embedding vector  $\mathbf{d}(z)$ 로 전환
  - Query  $x$ 가 입력되면 embedding vector  $\mathbf{q}(x)$ 로 전환
  - Maximum Inner Product Search (MIPS)를 이용해 top-K documents  $z_i$  탐색
  - FAISS(Facebook AI Similarity Search) 라이브러리 활용 (VectorDB)
- Generator: BART-large (encoder-decoder)
  - Input: Concatenate  $x$  &  $z_i$
- 최종 output: Marginalized된 하나의 sentence  $y$
- Training
  - Retriever training:  $\text{BERT}_d$  훈련 시 모든 document를 다시 indexing 해야함. → Query encoder( $\text{BERT}_q$ )만 fine-tuning  $\therefore$  query vector가 관련 document 쪽으로 더 강하게 향하도록 만들.
  - Generator training: BART 훈련
  - End-to-End: Loss  $\sum_j -\log p(y_j|x_j)$  계산 시, 신호가 generator를 거쳐 retriever 까지 전달
- Models: RAG-Token model & RAG-Sentence model  
Marginalization 방법에 따라 구분

$p_{\eta}$  : Retriever

$p_{\theta}$  : Generator

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

◦ RAG-Token

- 한 token당 각 retrieved document에 대해 단일 vocab distribution 생성
- K개의 retrieved documents에 대해 유사도 점수  $p_{\eta}(z|x)$ 를 따라 가중합 → 바로 최종 output 생성됨.

$$p_{\text{RAG-Token}}(y|x) = \prod_i^N p'_{\theta}(y_i|x, y_{1:i-1})$$

$$\text{where } p'_{\theta}(y_i|x, y_{1:i-1}) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

: 일반적인 beam search decoding

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

◦ RAG-Sequence

- 한 retrieved document당 하나의 완전한 문장을 생성함. → vocab distribution sequence
- K개의 output sequence probability를 유사도 점수  $p_{\eta}(z|x)$ 에 대하여 가중합 (marginalization)
- Decoding problem: 각 document마다 beam search를 통해 내놓은 문장 후보군이 다 다름.
  - Document  $z_1$  beam search 결과:  $y_1, y_2$
  - Document  $z_2$  beam search 결과:  $y_1, y_3$
  - $Y = \{y_1, y_2, y_3\}$
- Thorough decoding: 모든 beam 후보군  $Y$ 에 대해서 각 document에 forward pass하여  $p_{\theta}(y|x, z_i)$  계산 → 연산량 증가 ∴ Fast decoding 사용
- Fast decoding: 만약  $y$ 가  $x, z_i$ 로부터의 beam search에 나오지 않았다면  $p_{\theta}(y|x, z_i) \approx 0$ 으로 간주. Thorough decoding에 비해 성능의 큰 저하는 없음.

## 2. Experiments & Results

- Performances

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix [D](#) for further details.

|        | Model                           | NQ          | TQA               | WQ          | CT          |
|--------|---------------------------------|-------------|-------------------|-------------|-------------|
| Closed | T5-11B <a href="#">[52]</a>     | 34.5        | - /50.1           | 37.4        | -           |
| Book   | T5-11B+SSM <a href="#">[52]</a> | 36.6        | - /60.5           | 44.7        | -           |
| Open   | REALM <a href="#">[20]</a>      | 40.4        | - / -             | 40.7        | 46.8        |
| Book   | DPR <a href="#">[26]</a>        | 41.5        | <b>57.9</b> / -   | 41.1        | 50.6        |
|        | RAG-Token                       | 44.1        | 55.2/66.1         | <b>45.5</b> | 50.0        |
|        | RAG-Seq.                        | <b>44.5</b> | 56.8/ <b>68.0</b> | 45.2        | <b>52.2</b> |

- T5, DPR보다 좋은 성능
- Retrieved document에 없는 질문을 했을 때도 11.8% accuracy (Extracted model: 0% acc.)

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [\[4\]](#), FEVER-3 is [\[68\]](#) and FEVER-2 is [\[57\]](#) \*Uses gold context/evidence. Best model without gold access underlined.

| Model    | Jeopardy    |             | MSMARCO      |              | FVR3        | FVR2         |
|----------|-------------|-------------|--------------|--------------|-------------|--------------|
|          | B-1         | QB-1        | R-L          | B-1          | Label       | Acc.         |
| SotA     | -           | -           | <b>49.8*</b> | <b>49.9*</b> | <b>76.8</b> | <b>92.2*</b> |
| BART     | 15.1        | 19.7        | 38.2         | 41.6         | 64.0        | 81.1         |
| RAG-Tok. | <b>17.3</b> | <b>22.2</b> | 40.1         | 41.5         | 72.5        | <u>89.5</u>  |
| RAG-Seq. | 14.7        | 21.4        | <u>40.8</u>  | <u>44.2</u>  |             |              |

- BART에 비해 SotA(gold context를 prompt에 입력)에 가까운 성능
- Parametric & non-parametric memories work together

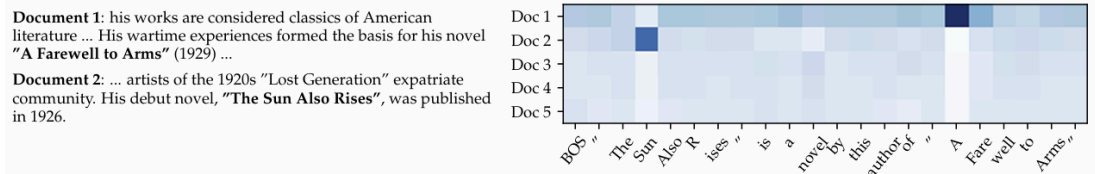
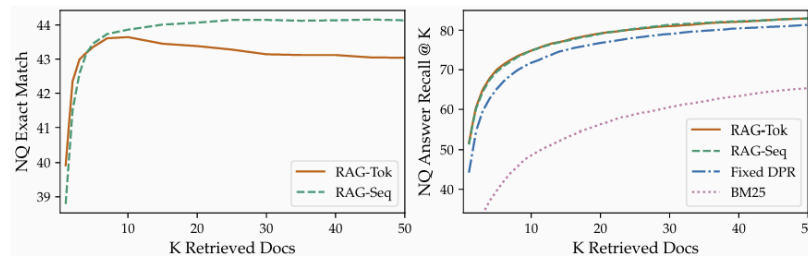


Figure 2: RAG-Token document posterior  $p(z_i|x, y_i, y_{-i})$  for each generated token for input "Hemingway" for Jeopardy generation with 5 retrieved documents. The posterior for document 1 is high when generating "A Farewell to Arms" and for document 2 when generating "The Sun Also Rises".

- "Hemingway"가 Answer인 Query를 생성하는 task
- 첫 번째(혹은 두 번째) token을 생성할 때 외부 지식(non-parametric memory)에 의존함.
- "The Sun", "A"까지만 봐도 내부 지식(parametric memory)으로 다음 token을 잘 생성함.
- Effect of # of Retrieving Documents



- RAG-Token: K=10일 때 peak
  - RAG-Sequence: 성능 일정 수준 유지
- 10개 이하의 document retrieval이면 충분함.

## Fine-Tuning vs. RAG

Title: Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs

Venue: EMNLP 2024

Link: <https://aclanthology.org/2024.emnlp-main.15/>

### ▼ Embeddings with a Single Pre-trained Encoder

Title: Text and Code Embeddings by Contrastive Pre-Training

Year: 2022

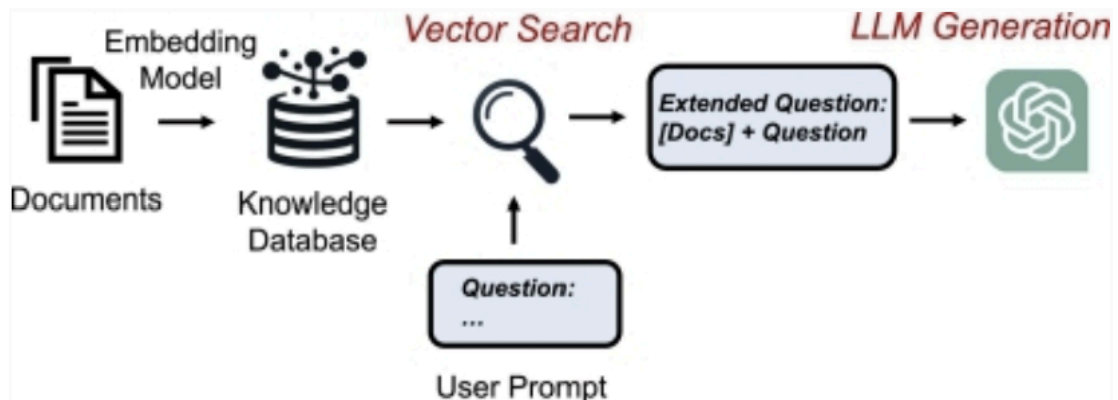
Link: <https://arxiv.org/abs/2201.10005>

Retrieval augmented generation (RAG) (Lewis et al., 2020) is a technique that expands LLMs' capabilities, especially in knowledge-intensive tasks, by using external knowledge sources. While the original formulation involved additional training per task, it has since been demonstrated (Neelakantan et al., 2022) that a pre-trained *embedding* model can achieve improved performance with no additional training involved.

- 대규모 contrastive training으로 linear-probe classification, text search, code search까지 한 번에 커버하는 encoder를 제안함.
- Positive pair: 이웃한 text 조각들
- 한 batch 내에 무작위 negative pair 수를 늘려야 함. → 큰 batch size가 중요

## 1. Methods

- Embedding Model  $\mathcal{M}_e$ : BGE-large-en (BERT-large 기반 영어 text embedding 모델. MTEB(Massive Text Embedding Benchmark)에서 SOTA를 달성.)



External document  $b \in \mathcal{B}_Q$

Given query  $q$ , embedding =  $\mathcal{M}_e(q)$  for encoder  $\mathcal{M}_e$

Retrieved top-K documents  $b_q = \{b_k\}_1^K$

New query  $\tilde{q} = \mathbf{b}_q || q$

최종 Output:  $\mathcal{M}(\tilde{q})$  for LLM  $\mathcal{M}$

- Knowledge-intensive Task: MMLU(Anatomy, Astronomy, Biology, Chemistry, Prehistory)
- Current Events Task: LLM 학습 cutoff 이후의 지식 (2023년 8-11월 미국에서 일어난 사건들의 Wikipedia)

- Fine-tuning: Unsupervised learning
  - 왜 Unsupervised learning? → QA 만들기가 어려움.
  - Next token prediction

## 2. Experiments & Results

Table 1: Results for the MMLU datasets described in Section 4.1 in terms of log-likelihood accuracy (Equation (4)).

| Task                       | Model      | Base model | Base model + RAG | Fine-tuned | Fine-tuned + RAG |
|----------------------------|------------|------------|------------------|------------|------------------|
| Anatomy (0-shot)           | Mistral 7B | 0.556      | <b>0.681</b>     | 0.570      | 0.659            |
|                            | Llama2 7B  | 0.393      | <b>0.489</b>     | 0.430      | <b>0.489</b>     |
|                            | Orca2 7B   | 0.607      | <b>0.637</b>     | 0.600      | <b>0.637</b>     |
| Anatomy (5-shot)           | Mistral 7B | 0.600      | <b>0.681</b>     | 0.622      | 0.674            |
|                            | Llama2 7B  | 0.467      | <b>0.563</b>     | 0.496      | 0.548            |
|                            | Orca2 7B   | 0.570      | 0.659            | 0.593      | <b>0.674</b>     |
| Astronomy (0-shot)         | Mistral 7B | 0.625      | 0.678            | 0.651      | <b>0.697</b>     |
|                            | Llama2 7B  | 0.401      | 0.467            | 0.487      | <b>0.520</b>     |
|                            | Orca2 7B   | 0.645      | <b>0.750</b>     | 0.651      | <b>0.750</b>     |
| Astronomy (5-shot)         | Mistral 7B | 0.658      | <b>0.724</b>     | 0.651      | 0.697            |
|                            | Llama2 7B  | 0.401      | 0.474            | 0.447      | <b>0.520</b>     |
|                            | Orca2 7B   | 0.664      | <b>0.763</b>     | 0.664      | 0.743            |
| College biology (0-shot)   | Mistral 7B | 0.681      | 0.757            | 0.701      | <b>0.764</b>     |
|                            | Llama2 7B  | 0.438      | <b>0.493</b>     | 0.458      | 0.465            |
|                            | Orca2 7B   | 0.583      | <b>0.639</b>     | 0.604      | 0.632            |
| College biology (5-shot)   | Mistral 7B | 0.722      | <b>0.778</b>     | 0.736      | 0.771            |
|                            | Llama2 7B  | 0.451      | <b>0.521</b>     | 0.424      | 0.479            |
|                            | Orca2 7B   | 0.604      | <b>0.660</b>     | 0.625      | 0.653            |
| College chemistry (0-shot) | Mistral 7B | 0.470      | <b>0.500</b>     | 0.490      | <b>0.500</b>     |
|                            | Llama2 7B  | 0.310      | 0.380            | 0.390      | <b>0.390</b>     |
|                            | Orca2 7B   | 0.370      | <b>0.440</b>     | 0.370      | 0.390            |
| College chemistry (5-shot) | Mistral 7B | 0.470      | <b>0.540</b>     | 0.500      | 0.500            |
|                            | Llama2 7B  | 0.370      | 0.380            | 0.360      | <b>0.390</b>     |
|                            | Orca2 7B   | 0.430      | <b>0.470</b>     | 0.370      | 0.380            |
| Prehistory (0-shot)        | Mistral 7B | 0.713      | <b>0.750</b>     | 0.719      | 0.731            |
|                            | Llama2 7B  | 0.448      | <b>0.481</b>     | 0.457      | 0.478            |
|                            | Orca2 7B   | 0.642      | <b>0.679</b>     | 0.673      | 0.673            |
| Prehistory (5-shot)        | Mistral 7B | 0.722      | <b>0.762</b>     | 0.725      | <b>0.762</b>     |
|                            | Llama2 7B  | 0.515      | 0.531            | 0.503      | <b>0.537</b>     |
|                            | Orca2 7B   | 0.664      | <b>0.698</b>     | 0.667      | 0.694            |

Table 2: Current events results. Models that were fine-tuned on the original dataset are labeled as *FT-reg*, while those trained on the dataset with multiple paraphrases are labeled as *FT-par*.

|            | Base model | Base model + RAG | FT-reg | FT-par | FT-reg + RAG | FT-par + RAG |
|------------|------------|------------------|--------|--------|--------------|--------------|
| Mistral 7B | 0.481      | <b>0.875</b>     | 0.504  | 0.588  | 0.810        | 0.830        |
| Llama2 7B  | 0.353      | <b>0.585</b>     | 0.219  | 0.392  | 0.326        | 0.520        |
| Orca2 7B   | 0.456      | <b>0.876</b>     | 0.511  | 0.566  | 0.820        | 0.826        |

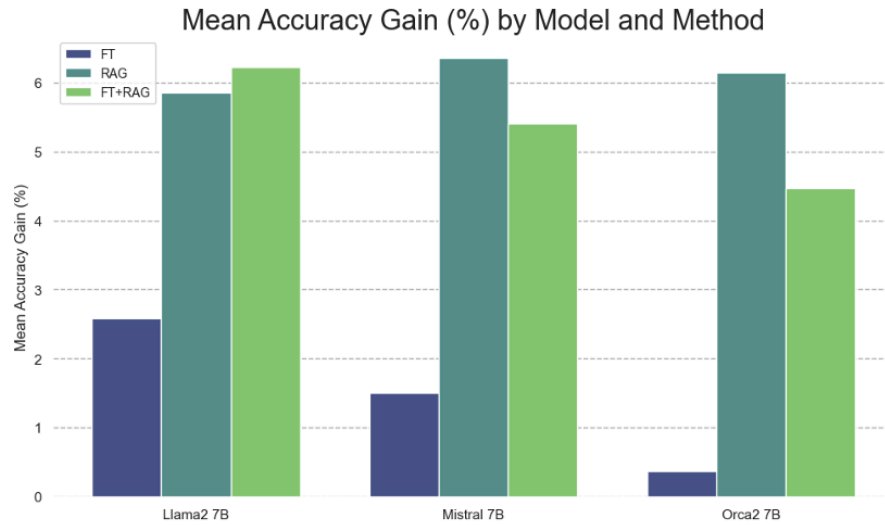


Figure 2: The relative accuracy gain (as explained in Equation (5)) for each knowledge-injection method, averaged (columnwise) across all experiments in Table 1.

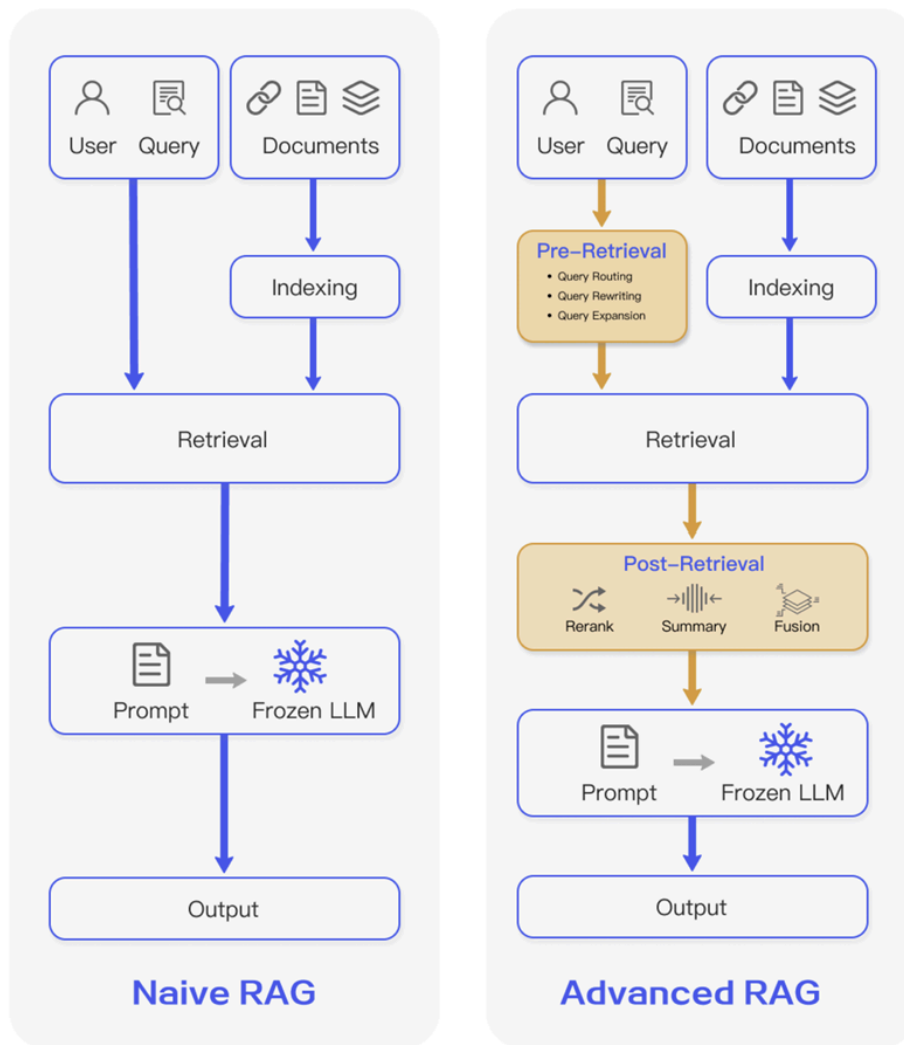
- Base model + RAG가 대체적으로 가장 우수한 성능을 보임.
- 왜 RAG > Fine-tuning?
  1. Contextual Relevance: FT는 학습한 지식은 모델의 parameter 전반에 걸쳐 퍼져있기에 질문을 받았을 때, 자기가 배운 것 중 무엇이 이 질문과 연관되는지 스스로 찾아야 하지만, RAG에서는 검색한 연관된 문서를 prompt에 바로 입력.
  2. Catastrophic Forgetting: FT 과정에서는 catastrophic forgetting이 발생하나, RAG에서는 모델의 inference 능력을 온전히 보존하면서 외부 지식만 활용 가능.
  3. Alignment Issue: 본 실험에서는 unsupervised learning만 했지만, supervised learning과 RL을 통해 추가로 FT를 개선할 수 있다.

## RAG Survey

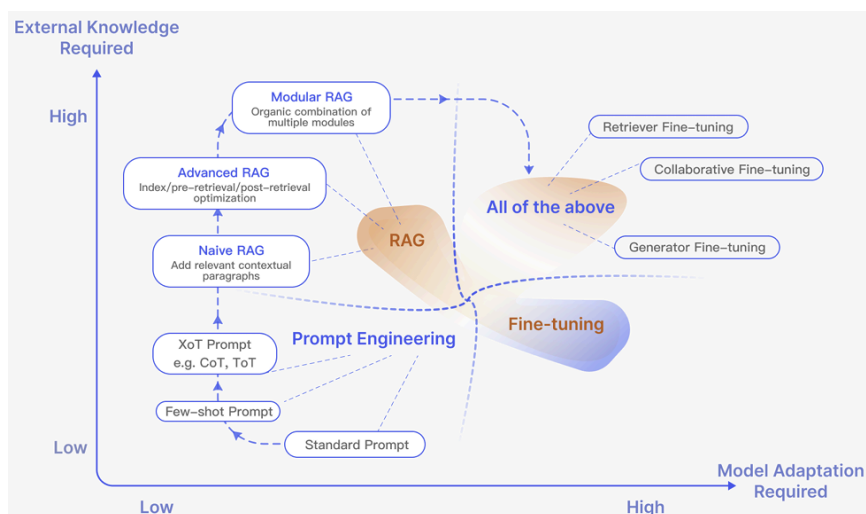
Title: Retrieval-Augmented Generation for Large Language Models: A Survey

Year: 2024

Link: <https://simg.baai.ac.cn/paperfile/25a43194-c74c-4cd3-b60f-0a1f27f8b8af.pdf>

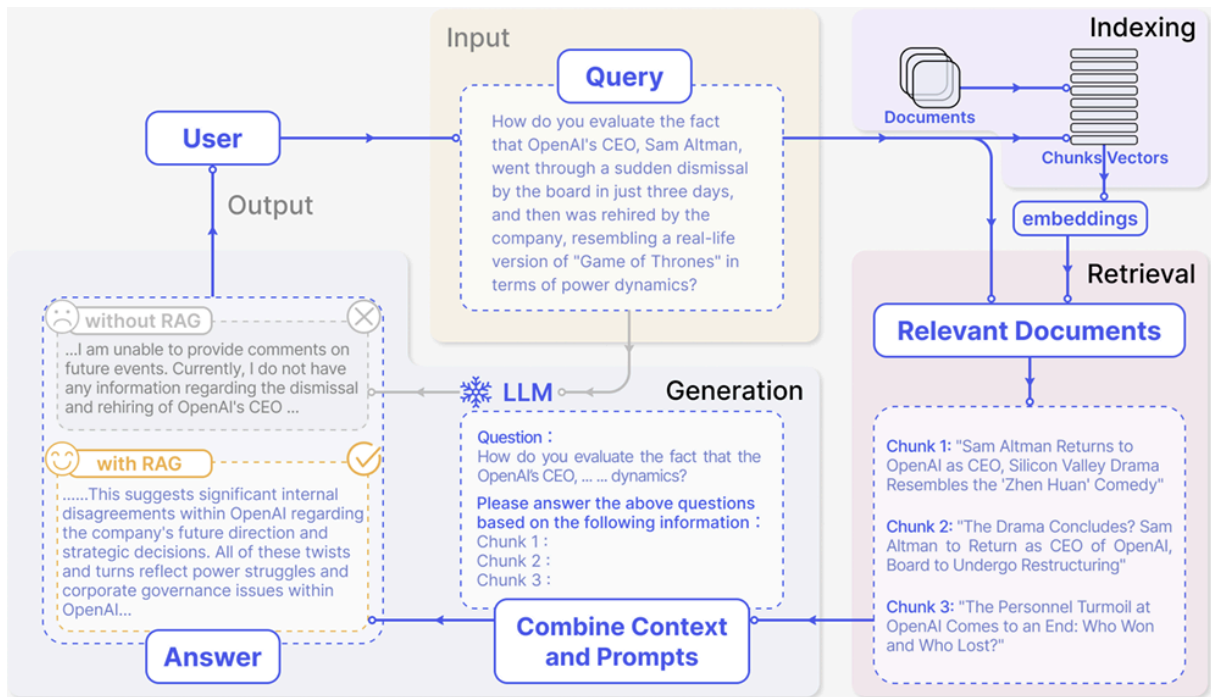


- RAG 연구 paradigm: Naive RAG, Advanced RAG



- RAG vs. Fine-Tuning vs. Prompt Engineering

## Naive RAG (Standard RAG)



- Workflows

1. Indexing

- PDF, HTML, Word, Markdown 같은 형식의 파일들을 통일된 text format으로 전환.
- Context limitation을 고려하여 text를 여러 chunk로 쪼갬.
- Embedding model을 사용하여 모든 chunk를 vector로 변환하고 VectorDB에 저장.
- 새 정보가 생길 때마다 VectorDB만 업데이트해주면 됨.

2. Retrieval

- Query가 들어오면, indexing 때와 동일한 embedding model을 사용해 vector로 변환.
- Query vector와 DB 내 가장 유사도가 높은 top-K chunk들을 탐색.

3. Generation

- 추출된 chunk들은 query와 concatenated되어 prompt로 구성됨.
- 새 prompt를 가지고 답변 생성.
- 대화 기록(history)이 있다면 이를 prompt에 통합해 문맥 유지.

- ⚠ Limitations

1. Retrieval Challenges: Retrieval 성능이 모델 성능을 크게 좌우함. 연관 없는/잘못된 chunk를 선택할 가능성.
2. Generation Difficulties: Prompt로 retrieved context를 주입해도 여전히 검색된 내용에 없는 거짓 정보를 지어낼 가능성(hallucination).
3. Augmentation Hurdles:

- 여러 retrieved chunk들을 통합하기가 쉽지 않음.
- 여러 sources에서 비슷한 정보가 추출될 경우 → 반복된 답변
- Documents 간 문체가 달라 결과물이 부자연스러울 수 있음.
- 답변을 생성하는 것이 아니라 단순히 검색된 chunk 내용을 반복하는 결과물을 내놓을 수도 있음.

## Advanced RAG

Retrieval 전후(pre/post)를 손봄으로써 Naive RAG limitations 개선

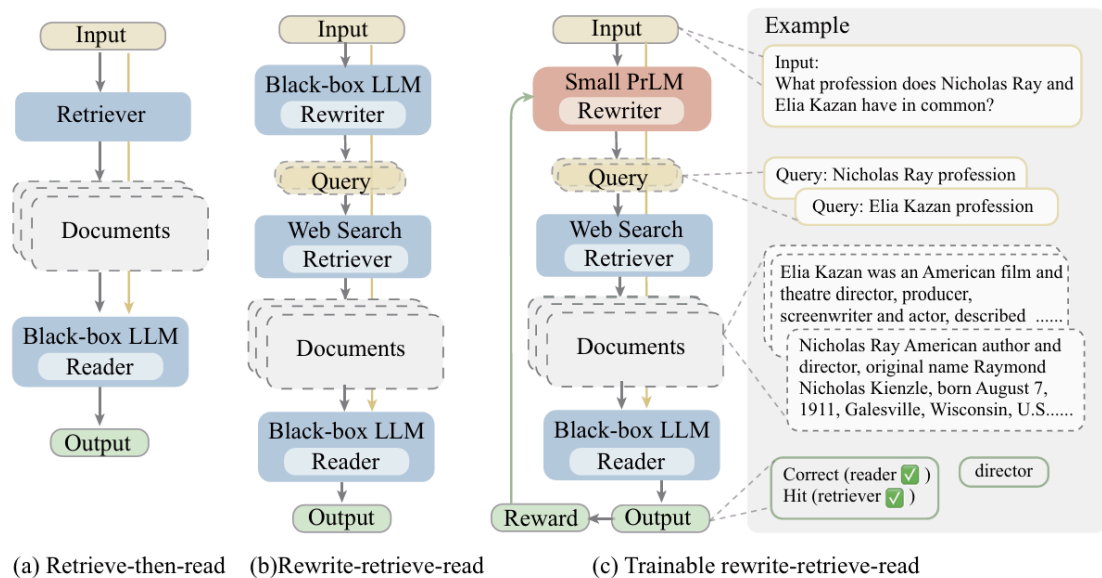
- Pre-retrieval Process

### ▼ Query Rewriting

Title: Query Rewriting for Retrieval-Augmented Large Language Models

Venue: EMNLP 2023

Link: <https://aclanthology.org/2023.emnlp-main.322/>



### 1. Introduction

- 기존 RAG (Retrieve-then-read): 사용자가 입력한 query가 검색하기 쉽지 않은 경우가 많음. 질문이 모호하거나 검색 엔진이 이해하기 어려운 언어로 되어 있음.
- Rewrite-retrieve-read: rewriter LLM 도입
  - Open-domain QA dataset  $D = \{(x, y)_i\}_{i=0}^N$
  - 1. Query rewrite: Original input  $x$ 를 rewriter LLM  $G_\theta$ 가 retrieval에 용이한 최적의 query  $\tilde{x}$ 로 재작성.
  - 2. Retrieve:  $\tilde{x}$ 와 연관된 context  $doc$  검색

3. Read: Original input  $x$ 와 추출한  $doc$ 을 이어붙인  $[doc, x]$ 를 reader LLM에 넣고, output  $\hat{y}$ 를 생성.
- Trainable RRR: Original input  $x$ 가 주어졌을 때, 작은 모델 rewriter가 큰 모델 reader가 잘 맞출 수 있는 검색어  $\tilde{x}$ 를 생성하는 법을 학습
  - Rewriter: Trainable, Reader: Frozen

## 2. Methods

- Trainable rewriter  $G_\theta$ : pre-trained T5-large
- Training  $G_\theta$  in 2 Stages: Warm-up & RL

### 1. Rewriter Warm-up

$$D_{Train} = \{(x, \tilde{x}) | \hat{y} = y\}$$

- Warm-up dataset: Reader LLM이 open-domain QA를 잘 맞췄을 때의 original question  $x$ 와 rewriter LLM이 재작성한 queries  $\tilde{x}$  pairs

$$\mathcal{L}_{warm} = - \sum_t \log p_\theta(\hat{\tilde{x}}_t | \tilde{x}_{<t}, x).$$

- Original question  $x$ 에 대해서 최적의 retrieval queries  $\tilde{x}$ 를 생성하도록 학습

### 2. RL

- Markov Decision Process  $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ 
  - State space  $\mathcal{S}$ : finite set limited by vocabulary and sequence length
  - Action space  $\mathcal{A}$ : vocabulary
  - Transition probability  $P$ : policy network  $G_\theta$ 에 의해 결정됨.
  - Reward function  $R$
  - Discount factor  $\gamma$
  - 매 step  $t$ 마다 action  $a_t$ 가 현재 state  $s_t = [x, \hat{\tilde{x}}_{<t}]$ 를 기반으로 next token  $\hat{\tilde{x}}_t$ 을 생성 → EOS token이 나오면 한 번의 episode가 종료됨.
- Warm-up이 끝난 rewriter  $G_\theta$ 의 initial policy model  $\pi_0$
- Reward Function

$$R(s_t, a_t) = R_{lm}(\hat{\tilde{x}}, y) - \beta \text{KL}(\pi_\theta || \pi_0)$$

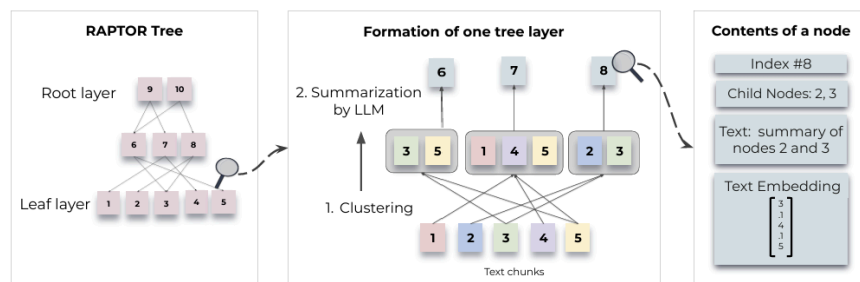
- $R_{lm}(\hat{x}, y)$ : reader가 내놓은 답  $\hat{y}$ 과 실제 정답  $y$  사이의 Exact Match(EM) 혹은 F1 Score → reader가 정답을 맞히면 rewriter가 보상을 받음.
- $KL(\pi_\theta || \pi_0)$ : 현재 모델  $\pi_\theta$ 가 처음 모델  $\pi_0$ 에서 너무 달라지지 않도록 penalty를 가함.

## ▼ RAPTOR

Title: RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

Venue: ICLR 2024

Link: <https://openreview.net/forum?id=GN921JHCRw>



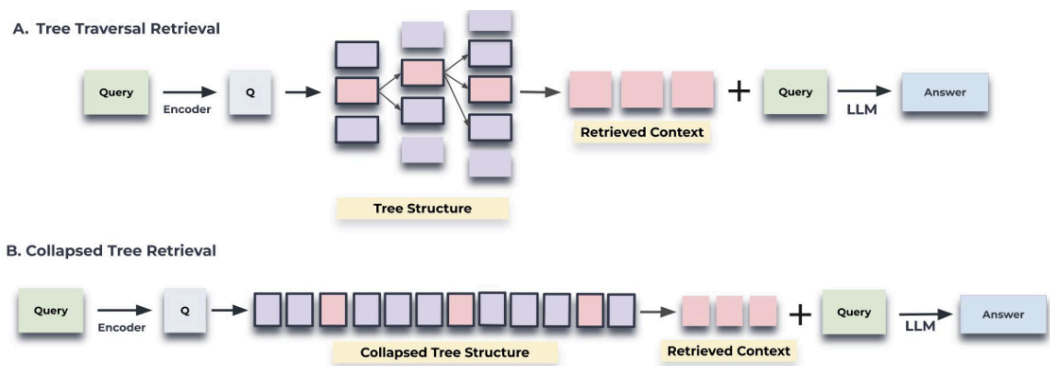
### 1. Introduction

- 기존 RAG 문제점: K개의 short & contiguous text chunk만 가져옴. → 문서 전체 맥락을 요구하는 query 해결 어려움. (e.g. "How did Cinderella reach her happy ending?")
- Indexing 및 retrieval에 tree 구조 제안
- 유사한 내용의 chunk들을 clustering으로 묶어서 summarization → 해당 요약들을 다시 묶어서 상위 summarization (tree 구조)
- Leaf node: 원본 chunk
- Parent node: children nodes의 요약
- Query에 맞는 상위 summarization & 하위 detail을 같이 retrieval

### 2. Methods

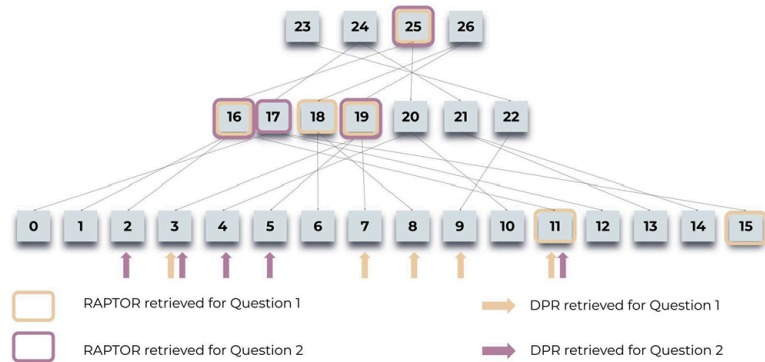
- Embedding model: SBERT
- Length 100의 short & contiguous text chunk로 자름. 문장이 100 token을 초과하면 잘린 문장 전체를 다음 chunk로 넘김.
- Soft Clustering
  - node가 여러 cluster에 동시에 속할 수 있음. ∴ 한 chunk가 여러 topic에 연관되어 있을 수 있음. → 여러 요약에 포함되는 게 자연스러움.

- UMAP: text embedding 고차원 → 거리 기반 metrics 불안정 → UMAP으로 차원 축소
- Gaussian Mixture Model(GMM) 알고리즘을 사용해 clustering
- GMM clustering 개수 =  $BIC = \ln(N)k - 2\ln(\hat{L})$   
 where  $N$ : # of text segments,  $k$ : # of 모델 parameter,  $\hat{L}$ : 모델의 maximized value of likelihood function
- Summerization
  - 한 clustering 내 node text들을 LLM(gpt-3.5-turbo)로 요약
  - 요약 후 길이가 평균적으로 children nodes 전체 길이의 28% (72% 압축률)
  - 요약 과정에서 4%의 hallucination 발생. 이 오류가 Parent nodes로 전달되지 않았음 & QA task에 영향을 주지 않았음.



- Tree Traversal Retrieval
  - Root에서 top-k nodes  $S_1$  뽑기 →  $S_1$ 의 children에 대해서 top-k node  $S_2$  뽑기 → 마지막  $d$  layer까지 반복
  - Query에  $S_1, S_2, \dots, S_d$ 를 concatenate
- Collapsed Tree Retrieval
  - 전체 node를 single layer set  $C$ 로 펼침.
  - $C$ 에서 top-k nodes 뽑기
  - Predefined maximum # of tokens 초과하지 않도록 유지하며 query에 nodes concatenate

### 3. Experiments & Results



- 신데렐라 이야기에 대한 두 질문 "What is the central theme of the story?" & "How did Cinderella find a happy ending?"이 주어졌을 때의 RAPTOR와 DPR의 selection

| Retriever        | GPT-3 F-1 Match | GPT-4 F-1 Match | UnifiedQA F-1 Match |
|------------------|-----------------|-----------------|---------------------|
| Title + Abstract | 25.2            | 22.2            | 17.5                |
| BM25             | 46.6            | 50.2            | 26.4                |
| DPR              | 51.3            | 53.0            | 32.1                |
| <b>RAPTOR</b>    | <b>53.1</b>     | <b>55.7</b>     | <b>36.6</b>         |

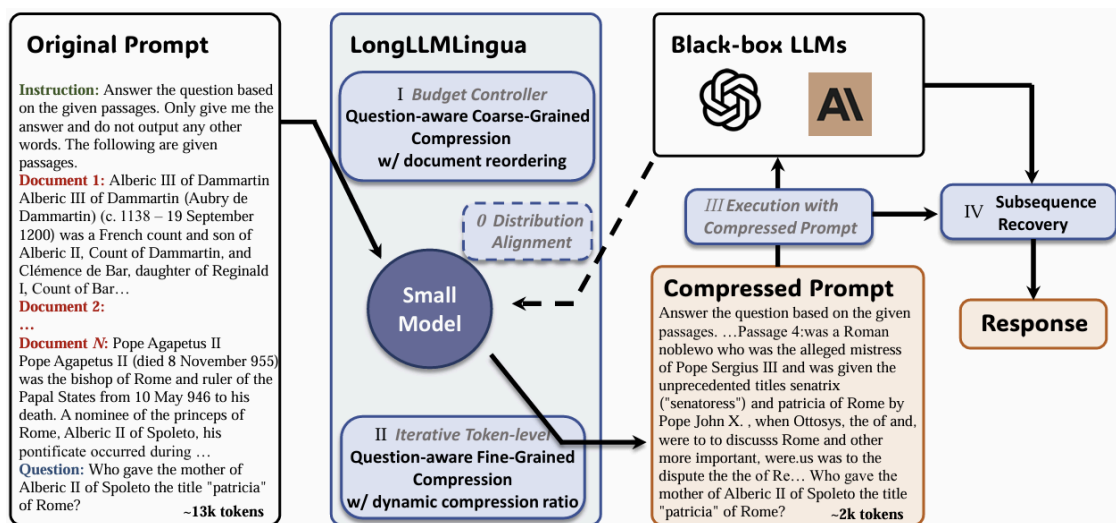
- RAPTOR의 QASPER dataset에 대한 성능이 BM25 & DPR보다 좋음.
- Post-retrieval Process

## ▼ Context Compressing

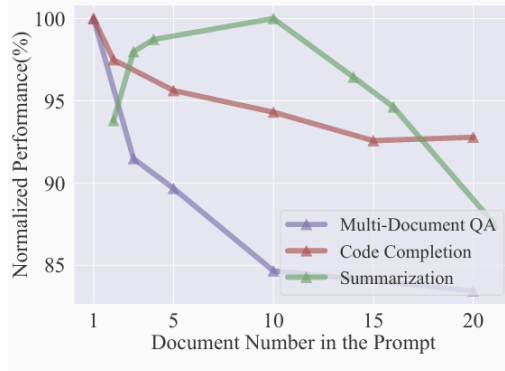
Title: LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression

Venue: ACL 2024

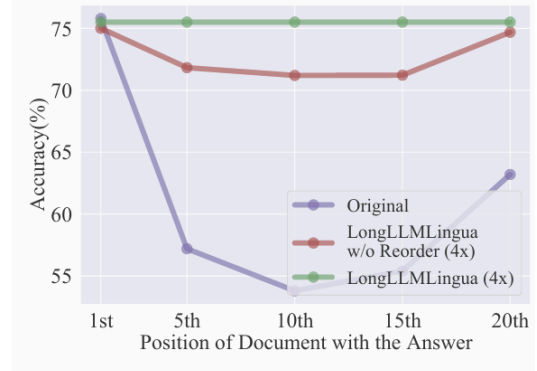
Link: <https://aclanthology.org/2024.acl-long.91/>



## 1. Introduction



(a) Performance v.s. Document Number



(b) Performance v.s. Key Information Position

$$\mathbf{x} = (\mathbf{x}^{\text{ins}}, \mathbf{x}_1^{\text{doc}}, \dots, \mathbf{x}_K^{\text{doc}}, \mathbf{x}^{\text{que}})$$

- Prompt  $\mathbf{x}$ : instruction,  $K$  documents, question 파트로 구성
- Long context의 문제점
  - Higher computational costs
  - 긴 prompt에 관련이 없거나 중복된 정보가 있어 LLM 성능 저하 유발 가능
  - LLM이 long context의 첫부분과 끝부분 정보는 잘 활용하지만, 중간에 포함된 정보는 놓치는 경향이 있음 ("lost in the middle")
- RAG와의 연결: retriever가 가져온 long context에서 질문에 실제로 필요한 정보만 남기도록 압축

## 2. Methods

$$\min_{\tilde{\mathbf{x}}} D_{\phi}(\mathbf{y}, \tilde{\mathbf{y}}) + \lambda \|\tilde{\mathbf{x}}\|_0$$

- Goal: 최적의 compressed prompt  $\tilde{\mathbf{x}}$  생성
  - $D_{\phi}$ : distance function (e.g. KL divergence)
  - $D_{\phi}(\mathbf{y}, \tilde{\mathbf{y}})$ : original prompt  $\mathbf{x}$ 에 대한 출력  $\mathbf{y}$ 과 compressed prompt  $\tilde{\mathbf{x}}$ 에 대한 출력  $\tilde{\mathbf{y}}$ 의 분포가 크게 달라지지 않도록 제어
  - $\|\tilde{\mathbf{x}}\|_0$ : 압축된 prompt의 token 수를 줄이도록 조절
  - $\lambda$ : 압축률

### 1. 어떻게 compressed prompt에서 중요한 정보의 비중을 높일까?

- Question-Aware Coarse-Grained Compression
  - Document-level: 여러 documents 중 버릴 documents 먼저 제거

$$\mathbf{x}_k^{\text{doc}} = \{x_{k,i}^{\text{doc}}\}_{i=1}^{N_k}$$

$$r_k = -\frac{1}{N_c} \sum_i^{N_c} \log p(x_i^{\text{que,restrict}} | \mathbf{x}_k^{\text{doc}}),$$

$$k \in \{1, 2, \dots, K\},$$

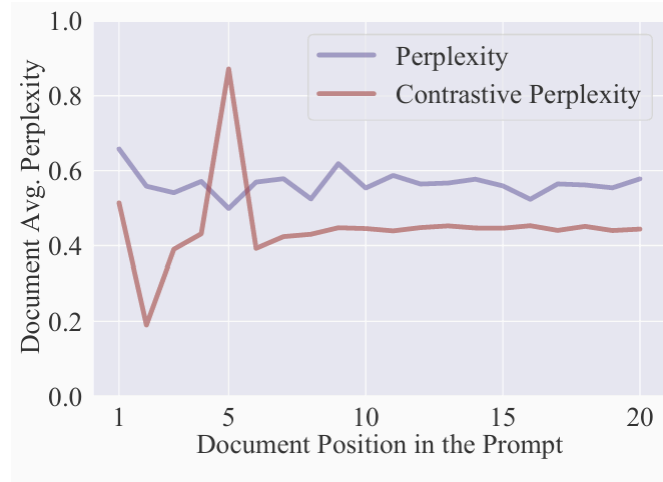
- $\mathbf{x}^{\text{que,restrict}} = (\mathbf{x}^{\text{que}}; \mathbf{x}^{\text{restrict}})$
- $\mathbf{x}^{\text{restrict}}$ : "We can get the answer to this question in the given documents."
- Score  $r_k$ : 단일 document  $\mathbf{x}_k^{\text{doc}}$ 만 주어졌을 때, 질문  $\mathbf{x}^{\text{que}}$ 에 잘 답할 수 있나?
- $r_k$  작을수록 LLM이 "이 document → 질문에 대한 답 생성"을 자연스럽게 느낌.

$$\{\mathbf{x}_i^{\text{doc}}\}_{i=1}^{K'}$$

- $K$ 에서  $K'$ 으로 필요한 document 수 감소
- Question-Aware Fine-Grained Compression
  - Token-level: 남은 documents 안에서 버릴 tokens 제거

$$s_i = \text{perplexity}(x_i | x_{<i}) - \text{perplexity}(x_i | x^{\text{que}}, x_{<i})$$

- $\{\mathbf{x}_k^{\text{doc}}\}_{k=1}^{K'}$ 의 각 token  $x_i$ 에 대해서 contrastive perplexity  $s_i$  계산
- Token  $x_i$ 의 예측이 질문을 알고 있을 때 더 쉬워지는가?



(b) Perplexity Distribution (5th)

- $\text{perplexity}(x_i|x_{<i})$ : 질문 없이 문맥상 next token이 얼마나 자연스러운지 (일반적인 예측)
- $\text{perplexity}(x_i|x^{\text{que}}, x_{<i})$ : 질문이 주어졌을 때 next token을 얼마나 확신 있게 예측하는지
- $s_i \approx 0$ : 질문이 있든 없든 예측 난이도 거의 동일
- $s_i \gg 0$ : 질문이 주어지면 예측이 훨씬 쉬워짐.

$$\begin{aligned}
 s_i &= \text{perplexity}(x_i|x_{<i}) - \text{perplexity}(x_i|x^{\text{que}}, x_{<i}) \\
 &= q(x_i) \log p(x_i|x^{\text{que}}, x_{<i}) - q(x_i) \log p(x_i|x_{<i}) \\
 &= q(x_i) \log \frac{p(x_i|x^{\text{que}}, x_{<i})}{p(x_i|x_{<i})}
 \end{aligned}$$

$$\begin{aligned}
 p(x^{\text{que}}|x_i, x_{<i}) &= \frac{p(x_i|x^{\text{que}}, x_{<i})p(x^{\text{que}})}{p(x_i|x_{<i})} \\
 &= p(x^{\text{que}}) \frac{p(x_i|x^{\text{que}}, x_{<i})}{p(x_i|x_{<i})}
 \end{aligned}$$

$$s_i \propto p(x^{\text{que}}|x_i, x_{<i})$$

- Token  $x_i$ 가 주어졌을 때 질문을 생성할 확률

2. 어떻게 middle에서의 정보 손실을 줄일까? ("lost in the middle")

$$\begin{aligned}
 (\mathbf{x}^{\text{ins}}, \mathbf{x}_1^{\text{doc}}, \dots, \mathbf{x}_{K'}^{\text{doc}}, \mathbf{x}^{\text{que}}) &\xrightarrow{r_k} \\
 (\mathbf{x}^{\text{ins}}, \mathbf{x}_{r1}^{\text{doc}}, \dots, \mathbf{x}_{rK'}^{\text{doc}}, \mathbf{x}^{\text{que}})
 \end{aligned}$$

- 중요한 정보가 long context의 중앙이 위치하면 성능 저하

- LLM이 long context에서 앞/뒤 정보에 더 민감하므로 훨씬 중요한 정보들을 앞쪽으로 모으자.
- Score  $r_k$ 에 대한 document index 오름차순 재배열

### 3. 어떻게 압축 비율을 조절할까? (Adaptive Budget)

- 문서마다 key information density가 다름.
- 질문과 매우 관련 있는 문서는 덜 압축 (문서 내 token을 덜 제거), 덜 관련된 문서는 더 압축 (문서 내 token을 많이 제거)
- Budget  $\tau \in [0, 1]$ : 남기는 token 비율. 1에 가까울수록 덜 압축, 0에 가까울수록 많이 압축
- Initial budget  $\tau^{\text{doc}}$ : 모든 documents  $\{\mathbf{x}_k^{\text{doc}}\}_{k=1}^{K'}$ 가 동일한 budget  $\tau^{\text{doc}}$ 을 가진 상태로 초기화

$$\tau_k^{\text{doc}} = \max\left(\min\left((1 - \frac{2I(r_k)}{K'})\delta\tau + \tau^{\text{doc}}, 1\right), 0\right)$$

- $\delta_\tau$ : 문서 간 budget 차이를 조절하는 hyperparameter
- $I(r_k)$ : 중요도 점수  $r_k$ 의 ranking index. 매우 중요한 문서의 경우  $I(r_k) = 0, 1, \dots$
- max & min:  $[0, 1]$ 로 clamping

### 4. 어떻게 key information을 유지할 수 있을까?

- 압축할 때 핵심 정보가 손실되지 않도록 하는 방법
  - e.g. Original tokens: "Barack Obama was born in Honolulu."
  - Token-level 압축: "Obama Honolulu"
- 생성하는 output  $y$ 에서 압축 prompt  $\tilde{x}$ 와 일치하는 부분을 찾아, 이를 다시 원본 prompt  $x$ 의 내용으로 복원

---

**Algorithm 1** Token-level Subsequence Recovery Algorithm

---

**Input:** The original prompt  $x$ ; the compressed prompt  $\tilde{x}$ ; the generation response of LLMs  $y$ .

```
1: Set the final response list  $y_{\text{rec}} = \phi$ , the left token index of
   subsequence  $l$  to 0.
2: while  $l < y.\text{len}()$  do
3:   if Substring  $y_l \in \tilde{x}$  then
4:     Find the longer substring  $\tilde{y}_{\text{key},l} = \{y_l, y_{l+1}, \dots, y_r\} \in \tilde{x}$ .
5:     Find the maximum common shortest subsequence
        $x_{i,j} = \{x_i, x_{i+1}, \dots, x_j\}$  in the original prompt  $x$ .
6:     Add the subsequence  $x_{i,j} = \{x_i, x_{i+1}, \dots, x_j\}$ 
       to the response  $y_{\text{rec}}$ .
7:     Set the left index  $l$  to  $r + 1$ .
8:   else
9:     Add the token  $y_l$  to the response  $y_{\text{rec}}$ .
10:    Set the left index  $l$  to  $l + 1$ .
11:   end if
12: end while
```

**Output:** The final response list  $y_{\text{rec}}$ .

---

- $y_{\text{rec}}$ : 복구된 최종 output
- $y$ 의 각 token  $y_l$ 에서 시작해서  $\tilde{x}$  내에 존재하는 가장 긴 연속된 token 묶음 (substring) 선택:  $\tilde{y}_{\text{key},l} = \{y_l, y_{l+1}, \dots, y_r\} \in \tilde{x}$
- 찾아낸  $\tilde{y}_{\text{key},l}$ 를 원본 prompt  $x$ 의 어떤 구간에 해당하는지 확인. Maximum common shortest subsequence 추출:  $x_{i,j} = \{x_i, x_{i+1}, \dots, x_j\} \in x$ 
  - e.g. Original prompt  $x$ : "...was awarded in 1901 to Wilhelm Conrad Röntgen of Germany..."
  - Compressed prompt  $\tilde{x}$ : "...awarded 1901 to Wilhelm gen..." → Wilhelm이란 gen token만 남거 사이 token들은 제거됨.
  - LLM의 답변  $y$ : "...Wilhelm gen..."
  - $x_{i,j}$ : "Wilhelm Conrad Röntgen"
- Token 치환 및 response 재구성: 해당되는  $x_{i,j}$ 를 최종 응답  $y_{\text{rec}}$ 에 추가
- 만약,  $y_l$ 부터 시작하는 substring이 없을 경우,  $y_{\text{rec}}$ 에 그대로  $y_l$  추가

## ▼ RankGPT

Title: Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents

Venue: EMNLP 2023

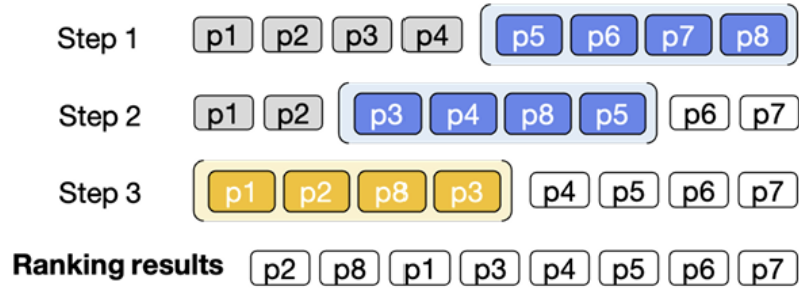
Link: <https://arxiv.org/abs/2304.09542>

- LLM이 passages를 중요도에 맞게 permutation generation하는 법을 학습

The following are passages related to query {{query}}  
 [1] {{passage\_1}}  
 [2] {{passage\_2}}  
 (more passages)  
 Rank these passages based on their relevance to the query.

[2] > [3] > [1] > [...]

- 큰 LLM(e.g. GPT-4)이 passage들 간의 질문과의 관련성 우선순위를 매김.



- Sliding Window: LLM context 제한 때문에 sliding window를 사용.
  - 뒤에서부터 중요도 순위 매김.
  - 위의 예시(step size 2)에서 관련 높은 상위 두 passsage가 다음 window에 다시 들어감.
  - 부분적으로 중요도 내림차순 정렬된 순위
- Permutation Distillation: 큰 모델(e.g. GPT-4)의 경우 cost 높음. → 작은 모델  $f_\theta$ 에 distillation

$$s_i = f_\theta(q, p_i)$$

- 학생 모델 (reranker): cross-encoder
- 교사 모델의 permutation 순위를 따라하는 것이 목표

$$\mathcal{L}_{\text{RankNet}} = \sum_{i=1}^M \sum_{j=1}^M \mathbb{1}_{r_i < r_j} \log(1 + \exp(s_j - s_i))$$

- M x M passages pairs
- $r_i < r_j$ : 교사 모델이 생성한 permutation 순위
- $l_{ij} = \log(1 + \exp(s_j - s_i)) = \log \sigma(s_i - s_j)$
- $s_i > s_j$ 가 되도록 최적화

- Passage 간 상대 순위가 맞도록 최적화

| Label   | Method        | DL19         | DL20         | BEIR (Avg)   |
|---------|---------------|--------------|--------------|--------------|
| ∅       | BM25          | 50.58        | 47.96        | 43.42        |
| ∅       | ChatGPT       | 65.80        | 62.91        | 49.37        |
| MARCO   | monoT5 (3B)   | 71.83        | 68.89        | 51.36        |
| MARCO   | DeBERTa-Large | 68.89        | 61.38        | 42.64        |
| MARCO   | LLaMA-7B      | 69.24        | 58.97        | 47.71        |
| ChatGPT | DeBERTa-Large | 70.66        | <b>67.15</b> | <b>53.03</b> |
| ChatGPT | LLaMA-7B      | <b>71.78</b> | 66.89        | 51.68        |